İlk zamanlar, veritabanlarında yalnızca şifre tutuluyordu. Bunun şöyle bir sıkıntısı var, eğer bir yolla veritabanı ele geçirilirse, şifreler otomatik olarak açıkta duruyordu. Bunun önüne geçebilmek için, önce hash algoritmaları kullandılar. Bu işleme hashing diyoruz. Hash algoritmaları, girişi kaç karakterli olursa olsun, fiks sayıda karakterli bir çıktı veren fonksiyonlardır. Ayrıca, geri döndürülemezler. Yani hash verildiğinde geri dönüş yapılamaz. Örneğin, MD5 çok bilinen ve çok kullanılmış bir hash algoritmasıdır. İki farklı değerin MD5 ile hash edilmiş haline bakalım.

Metin	**MD5 ile Hashlenmiş hali**
sifre	8cdee5526476b101869401a37c03e379
parola	8287458823facb8ff918dbfabcd22ccb

Bu düz şifre saklama olayından sonra, şifreleri hash ederek veritabanında tutmaya başladılar. Böylelikle, arada şifre transferi gerçekleşmiyordu. Kullanıcı şifresini girdiğinde, o şifreyi okuyan php dosyası, şifreyi md5 ile (ya da başka bir hash algoritması) hashleyerek, veritabanından, o kullanıcı için dönen hash ile karşılaştırıyordu.

Bu şekilde, veritabanı ele geçerse eğer, şifreleri aleni şekilde görmek engellenmişti. Ancak, çok uzun süre kullanıldı bu yöntem ve artık bazı insanlar, kendilerine bir md5 veritabanı yaratmaya başladılar.

Normalde, bu tarz hashlenmiş şeyleri kırmanın iki yolu vardır. Ya MD5 algoritmasında bir hata bulacaksınız, ya da tek tek her şifreyi deneyeceksiniz. Bunun yerine, bu arkadaşlar her şifreyi MD5 ile hashlediler ve bir veritabanında tuttular. Bunu da sayıca çok bilgisayar ile yeterince fazla süre içerisinde yaparsanız, birçok şifrenin otomatik olarak hash ini elinizde tutmuş olursunuz. Bunu yaptıkları ve MD5 şifrelerini kaydettikleri web siteleri bile var internette. Bunlara da hash-table, hash tabloları deniyor. Dolayısıyla, sadece MD5 algoritmasını kullanarak hash yapmanın da çok güvenli olmadığını söyleyebiliriz. Özellikle kolay seçilmiş şifrelerde: 123456, admin, password, sifre gibi.

Örneğin, MD5 ve Sha1 algoritmaları artık güvenli değil. An itibariyle SHA256 kullanılıyor. Geçmişe yönelik bir örnek vermek gerekirse, LinkedIn isimli web sitesi Sha1 kullanıyorlardı. 2012 de, veritabanına erişim sağladılar ve bu şifrelerin hashlenmiş halini yayınladılar. Çoğu kullanıcının şifreleri bu hashlerden geri döndürülebilmişti.

Salting

Bunun üzerine yeni bir uygulama yapmaya başladılar. Kriptoloji de nonce dediğimiz bir şey var. Nonce, bir kerelik kullanılan sayı demektir. Burada da, bu kavram "salt" adıyla kullanılıyor. Salt, bir kerelik kullandığımız bir kelime manasına geliyor. Amacı ise şu:

Normalde, biraz önce belirttiğimiz gibi, veritabanında şifrenin MD5 ile hashlenmiş hali duruyordu. Ancak bunun önüne geçmenin bazı yolları mevcut görüldüğü gibi. Bunu engellemek için, kullanıcının girdiği şifreye bir kelime eklendi (salt), ve bu yeni kelime hashlenerek veritabanında tutuldu. Böylelikle, 1- direkt olarak şifrenin hash hali durmuyor; 2- veritabanı ele geçerse eğer, aynı şifrenin bile veritabanında kaydı farklı hale geliyor.

Şöyle açıklayalım. İki farklı kullanıcı, aynı şifreyi kullanabilir. Hiçbir sitede bunun önüne geçilmez. Şu aralar, çok fazla değişken tanımlanıyor şifre oluştururken (karakter sınırı, büyük karakter şartı) ancak eskiden bu kadar yaygın değilken bu güvenlik önlemleri, çoğu insan kolay şifreleri seçiyordu. Şu anda olasılığı baya düşmüş olsa da, hala böyle bir şans mevcut.

Bu durumda, kullanıcı eğer çok bilindik bir şifre seçse bile; yani başkası ile aynı olsa bile, bu kullanıcılar farklı kullanıcı oldukları için, farklı bir "salt" ile ekleme yapılacak ve haliyle hash farklı olacaktır.

Salt + sifre	MD5
E1F53135E559C253 + 123456	b4d0cd97fa370307ee80594888021b20
84B03D034B409D4E + 123456	89444144ec426349600a48f50352c474

Yukarıdaki tabloda görüldüğü gibi, şifre aynı olsa bile; kullanıcılar farklı olduğu için farklı salt geliyor, ve veritabanında bu veriler farklı şekilde tutuluyor. Bu da bir güvenlik önlemi haline geliyor.

Ancak şunu unutmadan belirtmek gerek. Bu tarz önlemler, şifrenin tahminini güçleştirmez. Veritabanının ele geçme halinde, şifrelerin tahmin edilmesini zorlaştırır.

Yalnız, şunu bu aşamada belirtmek gerekiyor. Her kullanıcı için aynı salt u kullanırsanız, ve eğer sisteminize giriş yapılırsa, kaynak kodlarınızdan o salt u öğrenirler ve bu hiçbir işe yaramaz. Dolayısıyla, salt kullanırken her kullanıcı için başka bir salt yapmanız gerekiyor.

Aynı salt kullanmak hiçbir işe yaramaz, çünkü eğer salt öğrenilirse, şifre yerine "salt+şifre" kullanılır. Hiçbir fark yok.

Peki her kullanıcı için yeni bir salt oluşturursak, o zaman şahane bir güvenlik sağlar mıyız? Bunun için öncelikle, bu saltları nasıl belirleyeceğimizi konuşmak gerekiyor. Tamamen rastgele yapamayız, o kullanıcı ile ilgili bir şey kullanmalıyız ki bu kullanıcının şifresini tanımlarken kodda otomatik olarak bunu alalım. O zaman, yine bir algoritma kullanacağız demektir. Eğer sistemimize giriş yapılırsa, bu algoritma da ele geçecektir. Dolayısıyla, bu algoritma ele geçtiğinde, saldırgan yine her şeyi yapabilir. Tek fark, bu sefer tüm kullanıcılara değil, yalnızca tek kullanıcıya odaklanarak onun şifresini kırmaya çalışacak demektir. Biraz zorlaştırır işini, ancak tamamen güvenlikli diyemeyiz.

Buna ek olarak şunu belirtmek gerekiyor. Bu hash algoritmaları kasıtlı olarak hızlı çalışmaları için optimize edilmişler. Dolayısıyla, atak yapıldığında da bu algoritmalar hızlı çalıştığı için, saldırganların işi biraz kolaylaşıyor. Bunun önemini biraz sonra anlayacağız.

Bcrypt

Tüm bu sorunları ortadan kaldıran yeni bir sistem bu. Normalde, yukarıda genel hash algoritmalarından bahsettik. Bunların amacı, şifreleme değildir. Şifreleme başka, hash ise bambaşka bir eylemdir. İleride kriptoloji konularına ilgi duyarsanız detaylı öğrenebilirsiniz. Ancak bizler, bu hash algoritmalarını güvenlik amacıyla kullanmaya çalışıyoruz ve dolayısıyla biraz eksik kalıyoruz. Bcrypt ise, bu amaç ile yapılmış hash algoritmalarını kullanıyor. Yani, hem tek-taraflılar, hem de yavaş çalışıyorlar. Yavaş çalışmalarının avantaj olmadığını düşünebilirsiniz, ancak bu yavaş çalışma deneme sayısını limitliyor otomatik olarak.

Bcrypt algoritması, 100ms de çalışıyor. Sha1 den 10000 kat daha yavaş. Bu normal kullanıcı girişi yapılırken dikkat çekmese de, binlerce şifreyi hashleyerek deneme aşamasında ciddi yavaşlık sağlıyor.

Bunun yanında, her kullanıcı için ayrı salt oluşturarak, olası bir rainbow-table a karşı da koruma sağlıyor. Tabii, ne olursa olsun şifre seçimine de dikkat etmeli, mümkün mertebe fazla karakter ve değişik kombinasyonlar uygulamalıyız.